Separating the Credit Decision Stack

A controlled Qwen3.5-0.8B comparison of bundled and independently governed credit components

Abstract

Qwenderwriter combines a fixed-point credit scorecard, exact TreeSHAP tables, a demonstration policy and a language writer in one checkpoint. We reconstruct its deterministic build and mechanically extract its score and attribution engine, externalize credit rules into Open Policy Agent (OPA), and add an evidence graph, trusted-context checks and a sandbox execution gate. Both paths use the same unadapted Qwen3.5-0.8B weights, prompts, decoding settings and seven published requests. The comparison asks whether component separation changes credit responses, and separately tests the additional workflow controls.

Measured result	Finding
Credit response parity	7/7 complete assessment JSON responses identical
OPA policy parity	30,000 applications + 29 boundary cases; zero mismatches
Workflow extensions	12/12 expected control outcomes
Independent policy release	Decline becomes manual review; scorer hash and raw score unchanged

The evidence supports separation without changing the tested decisions. It does not establish better credit accuracy, lower cost, production readiness or superiority over the author's fine-tuned writer. The released fine-tuning stage was not reproduced.

Non-commercial derivative experiment. Original Qwenderwriter authors: Javier Mancilla and Tomas Tagliani, SquareOne Labs. This report is an independent implementation comparison, not a peer-reviewed publication.

1. Architecture and research question

The original already separates deterministic arithmetic from language generation. Its architectural choice is to ship these components under one checkpoint identity. Our question concerns independent governance and deployment artifacts, not whether an LLM should calculate credit decisions.

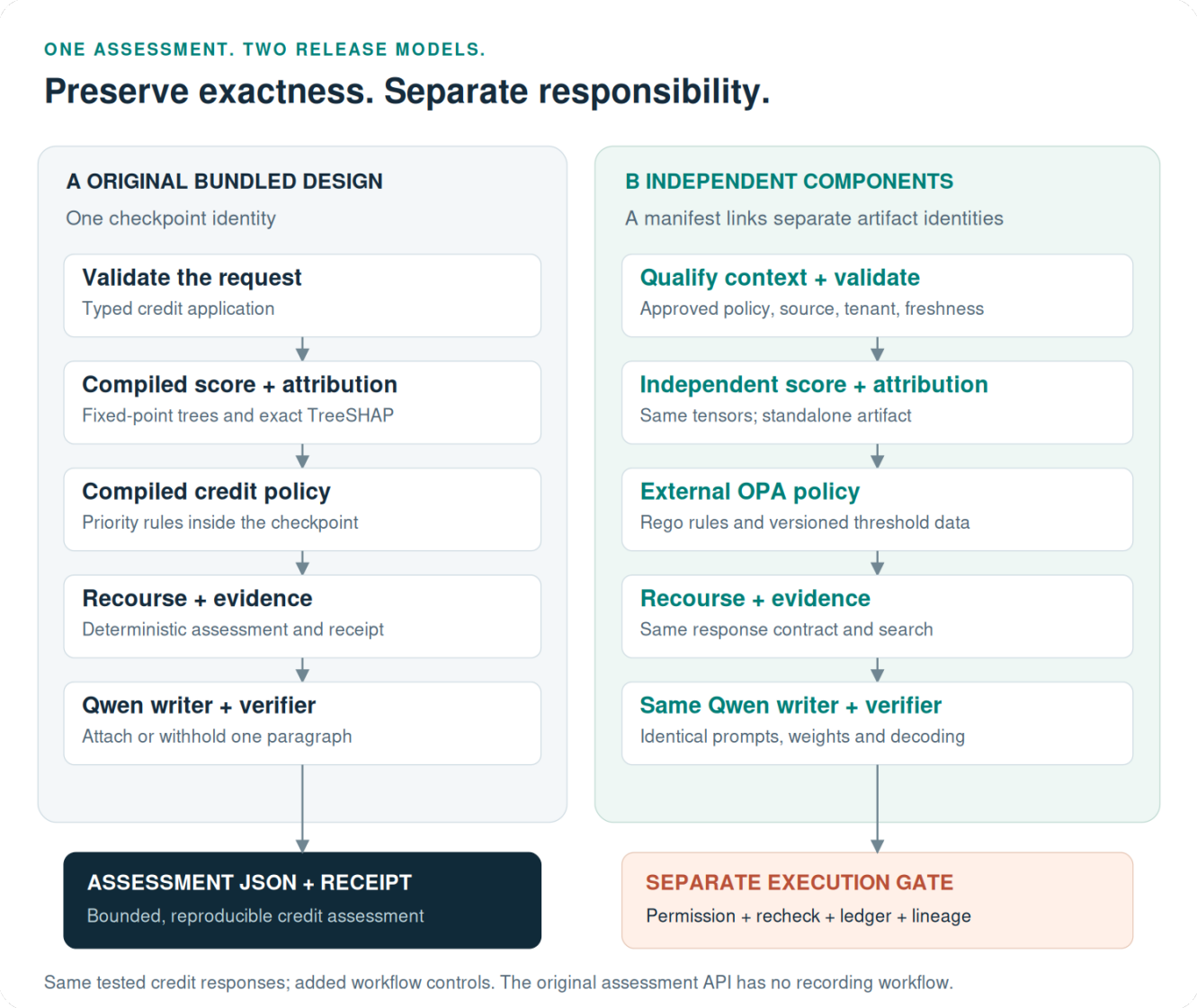


Figure 1. Left: original source design. Right: implemented component refactor. The right side is a modular Python application with a separate OPA process; it is not a deployed microservices system.

This independent credit architecture uses source and version relationships to determine applicable evidence; policy is evaluated explicitly; the writer receives fixed facts; an application gate separately checks authority before a sandbox write. The graph here is a minimal policy-approval and decision-lineage graph, not an enterprise ontology.

2. What changed and what stayed fixed

Layer	Original path	Separated path
Input contract	Upstream 23-field validator	Verbatim validator retained
Score / attribution	Compiled checkpoint buffers	Identical tensors in independent safetensors file
Credit policy	CheckpointDemoPolicy tensor rules	Independent feature calculations + Rego outcome rules
Language	Qwen writer inside original class	Same Qwen callable behind writer adapter
Explanation checks	Upstream licensed-vocabulary verifier	Same verifier, unchanged for comparison
Recourse	Upstream candidate search, up to 3 moves	Same search, rescored through external policy
Evidence context	Typed application and compiled receipt	Additional source, tenant, policy-approval and freshness checks
Execution	No external write workflow	Authorization, context recheck and SQLite sandbox ledger

Mechanically extracted arithmetic

The deterministic source classes and score verification methods are extracted with Python AST tooling. The language-model superclass and constructor are removed; arithmetic methods remain unchanged. A test verifies that the detached component loads no Transformers module and instantiates no language parameters. The extraction is credited and remains under the original non-commercial licence.

Policy ownership

The 15 demonstration credit triggers and priority outcome selection are implemented in Rego. Numerical feature calculations are independent NumPy code. Thresholds, development bounds and calibration coefficients live in policy.json. Legacy score grading and its raw-margin binary threshold are retained as scorer metadata for exact response parity; they are distinct from the demonstration credit-policy outcome.

A correction to the initial critique

The original paper explicitly states that policy can be recompiled without retraining the writer. Our contribution is an independently versioned policy artifact and runtime boundary, rather than avoiding an inherently required retraining step.

3. Controlled experimental method

Variable	Configuration
Upstream source	8d6015075d421eb1102db6dcc48a1caa62f84828
Common writer	Qwen/Qwen3.5-0.8B; unadapted base weights
Qwen revision	2fc06364715b967f1860aea9cf38778875588b17
Decoding	Greedy; thinking disabled; max_new_tokens = 320; seed = 20261102
Requests	Seven JSON requests shipped by the upstream repository
Generation	Both arms actually generate each applicable paragraph; no response cache
Hardware	CPU execution, four Torch threads; no GPU used
Policy runtime	OPA 1.22.0 via local subprocess
Software	Python 3.12.14; Torch 2.15.0+cu132; Transformers 5.22.1; XGBoost 3.1.2

Build and numerical evidence

The original training search and frozen data splits were reused. The reconstructed scorer achieves ROC-AUC 0.777889 and KS 0.436571, matching the paper's rounded 0.778 and 0.437. Native-to-compiled parity has zero sign inversions over 30,000 rows and zero ROC-AUC/KS difference. TreeSHAP compilation produces 344,106 regions and zero fixed-point local-accuracy residual.

A compatibility repair restored feature names and integer types omitted from the saved booster by the installed XGBoost/pandas combination. Tree parameters were not changed. Exact source, model and OPA identities are recorded in models/model_manifest.json, together with dependency versions and build logs.

The policy comparison uses the same compiled margins in both arms, with each of the ten returned tensor fields compared exactly: calculations, flags, risk indices and one-hot bands, triggers, outcome indices and one-hot outcomes, final reasons, raw model probability and frozen calibrated probability.

The author's LoRA-adapted writer and its reported 98.1% verification rate are outside this reproduction. Substituting the same base writer on both sides is a controlled architecture ablation, not a reproduction of that released writer.

4. Paired credit responses

Same test request	Outcome / probability	Paragraph	JSON parity
approval with controls	AC / 5.8%	VERIFIED	Identical
decline close to approval	DD / 69.9%	VERIFIED	Identical
invalid input missing field	II / 84.3%	WITHHELD	Identical
malformed override attempt	OVERRIDE_ATTEMPT / -	NO LM CALL	Identical
malformed string value	MALFORMED_TYPE / -	NO LM CALL	Identical
manual review not behavioural	MR / 10.6%	WITHHELD	Identical
standard approval	AS / 32.5%	VERIFIED	Identical

All 7 evidence strings, prompts and complete assessment JSON objects match. There were 10 actual writer calls: five applicable requests per arm; the two malformed requests never invoke Qwen. Three of five generated paragraphs pass verification in each arm. Two are withheld in each arm, while the deterministic assessment remains available.

Why an invalid-input case still invokes the writer

The file named invalid_input_missing_field uses an explicit null, which the upstream router accepts as a policy input. It becomes outcome II (invalid input) rather than a malformed-envelope refusal. This behavior is preserved, including the attempted and withheld paragraph. A physically absent required key is a different malformed-request class.

Latency interpretation

Case	Bundled seconds	Separated seconds
approval with controls	16.31	17.32
decline close to approval	19.56	18.44
invalid input missing field	26.62	25.77
manual review not behavioural	17.91	20.18
standard approval	14.24	13.90

These are single observations in fixed arm order, including generation and, on the separated path, context qualification. They are not randomized performance benchmarks. OPA process startup, hashes and graph checks add work; no speed or cost advantage is claimed.

5. Actual language responses

The paragraphs below were generated independently on each path and matched exactly. They are raw experimental wording, retained without editorial correction. Full requests, prompts, raw generations, verifier results and JSON responses are in results/paired_responses.json.

Approval With Controls | 5.8%

The credit analyst determines that the case is approved with controls based on a soft decision rule that identifies a rising balance trend as the primary active factor, which is the first level in the priority hierarchy. The model assigns a probability of 5.8% to the outcome, placing it within a low risk band. The main risk-increasing driver is the credit limit amount, while the risk-decreasing driver is the most recent repayment status. The calculations show a current utilization of 60.9% and a max utilization of 71.07%, with a payment coverage of 22.74% and a balance trend of 54.84%. A data quality flag indicates that a repayment code is undocumented in the UCI source. The case is very close to the next better outcome, which is standard demonstration approval, and the behavior cannot change it. The probability is a benchmark association, not a cause and not a guarantee.

Decline Close To Approval | 69.9%

The credit analyst determines that the demonstration decline outcome code DD is the correct decision for the case, which is controlled by the severe delinquency rule. The model assigns a probability of 69.9% to the demonstration decline, placing the case in an elevated risk band. The primary risk-increasing driver is the most recent repayment status, while the risk-decreasing driver is the payment amount three months ago. The active rules for hard severity are severe delinquency, and the controlling rules for review are multiple soft controls requiring review. The calculations show a current utilization of 58.93% with a max of 62.67%, indicating a significant increase in risk. The data quality flag indicates that a repayment code is undocumented in the UCI source. The case is very close to the next better outcome of approval with controls, and the behavior cannot change it. The probability is a benchmark association, not a cause and not a guarantee.

Standard Approval | 32.5%

The credit analyst determines that the standard demonstration approval decision is active and the policy allows standard approval without hard, review, or soft rules. The model assigns a probability of 32.5% to the outcome, placing the case within a moderate risk band. The primary risk-increasing driver is the most recent statement balance, while the risk-decreasing driver is the payment amount three months ago. The active calculation metrics show zero utilization and zero payment coverage, indicating no active risk control mechanisms are engaged. A data quality flag indicates that a repayment code is undocumented in the UCI source, which may affect the reliability of the underlying data. The case is currently at the best status and is not expected to change based on the current behavior. The probability serves as a benchmark association rather than a cause or guarantee for the outcome.

Verification is limited to the upstream gate's checks. Matching allowed numbers, labels and rule phrases does not establish comprehensive semantic correctness or suitability for customer communications.

6. Enterprise workflow extension tests

These cases use the actual approval-with-controls credit request and separated assessment. They test additional workflow controls. The upstream assessment API does not implement these controls or a write tool, so its capability is recorded as not implemented; it is not counted as making unsafe writes.

Scenario	Separated response	Result
valid record	RECORDED	Pass
stale policy	STALE POLICY	Pass
stale evidence	STALE EVIDENCE	Pass
untrusted source	UNTRUSTED SOURCE	Pass
wrong tenant	CROSS TENANT EVIDENCE	Pass
unprivileged principal	ASSESSMENT NOT AUTHORIZED	Pass
assess only principal	WRITE NOT AUTHORIZED	Pass
missing approval	MISSING POLICY APPROVAL	Pass
changed request	CONTEXT CHANGED BEFORE WRITE	Pass
policy changed in flight	CONTEXT CHANGED BEFORE WRITE	Pass
retry same write	ALREADY RECORDED	Pass
retry conflicting write	IDEMPOTENCY CONFLICT	Pass

Execution checks

Final verification also passes all 43 original repository tests (plus six subtests) and all 17 added component/governance tests. Original artifact tests run with offline model loading and explicit Torch thread limits; raw test logs are included.

Trusted context binds the request hash, current policy, component hashes and registry version. Before a sandbox write the application qualifies the live context again. A changed request, registry version or live component identity is rejected. Role checks distinguish assessment permission from recording permission. A SQLite primary key makes identical retries idempotent and conflicting retries fail.

The approval graph, source labels, evidence ages, tenant identities and role registry are synthetic trusted fixtures. They are not an implemented bank IAM, live bureau integration, signed policy registry or independently verified source system.

7. Policy releases, strengths and costs

An independent release experiment

A second synthetic policy deliberately relaxes the two severe-delinquency thresholds. For the published decline example, the outcome changes from DEMO_DECLINE to MANUAL_REVIEW while the raw fixed-point margin remains 842436674 (69.9% raw model probability). The scorer/attribution artifact SHA-256 remains identical, and no Qwen model is loaded. The release changes policy behavior; it is intentionally outside the parity comparison.

Claim	Evidence and qualification
Independent policy lifecycle	Separate Rego/data artifact; policy-only outcome change measured. Synthetic local approval, not institutional approval.
Independent scoring	Detached runtime instantiates no Qwen weights and loads no Transformers module.
Evidence applicability	Minimal graph and source/version/freshness/tenant checks reject declared invalid contexts.
Execution control	Application gate rechecks context and permissions before a sandbox record.
Audit reconstruction	Decision nodes link policy and evidence; events include timestamp, content hashes and previous-event hash.
Costs of separation	More interfaces, deployment identities, failure modes, policy-code maintenance and orchestration checks. OPA outages fail closed.

Neither packaging choice is universally superior

A single checkpoint offers convenient offline distribution, a compact release identity and fewer operational components. Separate artifacts support different owners and update cycles but require a coherent release manifest and integration tests. A graph or an OPA process alone cannot supply trustworthy evidence, resolve ambiguous policy or establish lawful lending behavior.

Our demonstrated strength is preserving bounded assessment behavior while making governance boundaries explicit and independently testable. We did not demonstrate better underwriting, cheaper inference, higher explanation quality or production-scale reliability.

8. Limits, reproducibility and references

Limits that affect interpretation

Seven paired requests are a small regression set. The 30,000-row result establishes equivalence of this policy implementation on the benchmark, not universal equivalence on all signed-64-bit inputs. Large extreme inputs may inherit upstream arithmetic overflow risks. The 29 boundary cases cover selected risk, delinquency, utilization and null boundaries, not every combination.

The writer is unadapted, English only, and has five generated test requests per arm. We retained the original explanation gate, including its known number-to-label semantic gap. We retained the bounded recourse search and its non-causal interpretation. No temporal validation, fairness assessment, regulatory certification, live lending action, distributed deployment or production access isolation was performed.

The event graph is in memory and its hashes are not signatures. The SQLite ledger can be persisted, but this prototype does not provide a production immutable audit store or multi-party approvals. Evidence freshness and identity assertions must come from authenticated enterprise integrations before real deployment.

Reproduction and deliverables

The repository contains the rewritten components, native XGBoost model weights, policy bundles, pinned model identities, seven fixtures, exact response logs, policy parity output, control cases, tests, report and editable architecture diagrams. The larger model bundle contains the rebuilt original-layout base checkpoint plus the detached scorer/attribution artifact. The Qwen weights are a common base, not the author's fine-tuned release.

Run scripts/reproduce.py to fetch the pinned upstream source and official Qwen snapshot, rebuild both deterministic layouts, run the paired generation and execute the control tests. A local base-model directory can be supplied to avoid another download. scripts/demo.py runs the separated credit workflow without Qwen when only structured output is needed.

Primary references

1. Mancilla, J.; Tagliani, T. (2026). Qwenderwriter. Original paper, sections 4-7. github.com/maximer-v/qwenderwriter/blob/main/paper/Qwenderwriter.pdf

2. Qwenderwriter source, commit 8d6015075d421eb1102db6dcc48a1caa62f84828. Original architecture, model card, policy, code and licence. github.com/maximer-v/qwenderwriter

3. Qwen team. Qwen3.5-0.8B official model and Apache-2.0 licence. huggingface.co/Qwen/Qwen3.5-0.8B; pinned revision in model manifest.

4. Open Policy Agent. Official documentation: decision-making is separated from enforcement; structured inputs and outputs. openpolicyagent.org/docs. Runtime version 1.22.0.

5. Local evidence: results/summary.json; paired_responses.json; policy_parity.json; governance_cases.json; policy_release.json; lineage_graph.json; build and test logs.